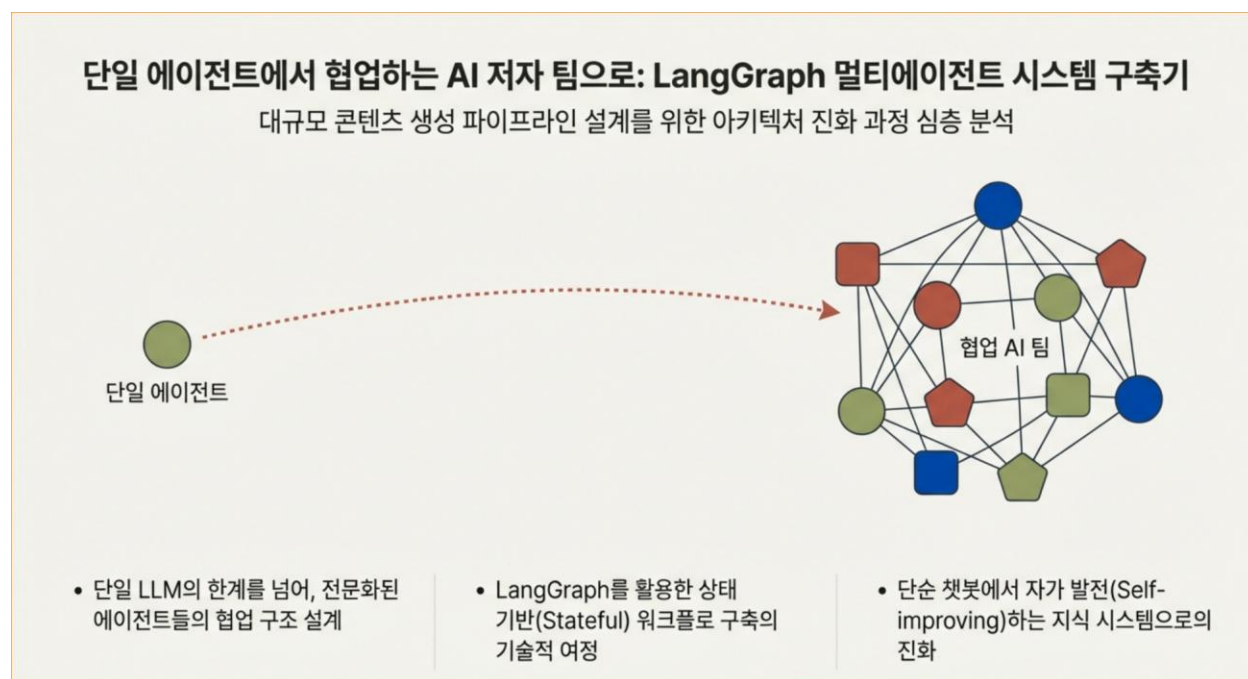


LangGraph 기반 자율형 멀티에이전트 시스템 아키텍처 및 구현 : 대규모 콘텐츠 생성 파이프라인 구축



1. 서론: 멀티에이전트 시스템의 패러다임 전환과 LangGraph의 역할

현대 인공지능 기술의 지평은 단일 대규모 언어 모델(Large Language Model, LLM)이 모든 작업을 처리하는 모놀리식(Monolithic) 접근 방식에서, 전문화된 기능을 수행하는 다수의 에이전트가 협력하여 복잡한 문제를 해결하는 멀티에이전트 시스템(Multi-Agent System, MAS)으로 급격히 이동하고 있다. 이러한 변화는 복잡한 인지 작업이 요구되는 콘텐츠 생성, 소프트웨어 엔지니어링, 그리고 정교한 데이터 분석 영역에서 특히 두드러진다. 단일 에이전트는 컨텍스트 윈도우의 제한, 도구 선택의 모호성, 그리고 단일 프롬프트 내에서의 역할 혼재로 인해 복잡도가 증가할수록 성능 저하 및 환각(Hallucination) 현상을 겪기 쉽다. 반면, 멀티에이전트 아키텍처는 인간 조직의 분업화 원리를 차용하여, 각기 다른 전문성을 가진 에이전트들이 상호작용하며 작업을 수행함으로써 이러한 한계를 극복한다.

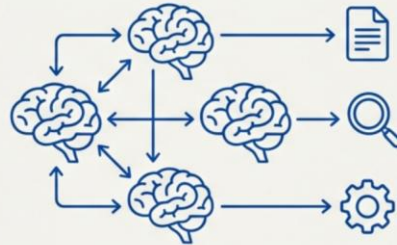
왜 모놀리식 LLM은 복잡한 작업에 실패하는가?

모놀리식 접근의 한계



- **컨텍스트 윈도우의 제약:** 작업이 복잡해질수록 필요한 모든 정보를 단일 컨텍스트에 담기 어려움.
- **도구 선택의 모호성:** 여러 도구 중 어떤 것을 언제 사용해야 할지 결정하는 데 어려움을 겪음.
- **역할 혼재와 성능 저하:** 단일 프롬프트 내에서 사용자 응대, 데이터 분석, 콘텐츠 생성을 동시에 수행하며 발생하는 환각(Hallucination) 현상.

패러다임의 전환: 멀티에이전트 시스템(MAS)



- **인간 조직의 분업화 원리:** 각기 다른 전문성을 가진 에이전트들이 상호작용하며 복잡한 문제를 해결.
- **명확한 역할 분담:** 각 에이전트는 하나의 잘 정의된 책임을 가짐으로써 성능과 신뢰성을 극대화.

사용자의 요구에 따라 책의 목차를 작성하고, 관련 자료를 조사하며, 최종적으로 콘텐츠를 생성하는 'AI 저자 팀(AI Author Team)' 구축 프로젝트를 중심으로, LangGraph 프레임워크를 활용한 멀티에이전트 시스템의 설계 및 구현 과정을 심층적으로 분석한다. 특히, 단순한 챗봇(Communicator)에서 시작하여, 목차 생성 전문가(Content Strategist), 중앙 관리자(Supervisor), 그리고 외부 지식을 탐색하는 연구원(Web/Vector Search Agent)으로 시스템이 진화해 나가는 과정을 기술적으로 해체하고 재구성한다. 이 과정에서 LangGraph의 핵심 개념인 상태 그래프(StateGraph), 노드(Node), 엣지(Edge), 그리고 조건부 라우팅(Conditional Routing)의 기술적

메커니즘을 상세히 규명하며, Pydantic 을 활용한 구조화된 데이터 통신과 RAG(Retrieval-Augmented Generation) 기반의 지식 확장 전략을 포괄적으로 다룬다.

1.1 LangGraph 프레임워크의 아키텍처 철학

LangGraph 는 LangChain 생태계의 확장이자, 에이전트 간의 복잡한 상호작용을 '그래프(Graph)'로 모델링할 수 있게 해주는 오케스트레이션 프레임워크이다. 기존의 선형적인 체인(Chain) 구조는 순차적인 작업 처리에는 유용하나, 반복적인 루프(Loop), 조건부 분기, 그리고 상태 유지(Statefulness)가 필수적인 에이전트 워크플로를 구현하는 데에는 한계가 있었다. LangGraph 는 Google 의 Pregel 시스템에서 영감을 받은 메시지 전달(Message Passing) 알고리즘을 기반으로 하여, 노드 간의 데이터 흐름을 명시적으로 제어하고 애플리케이션의 상태를 영속적으로 관리할 수 있는 기능을 제공한다.

LangGraph: 순환과 분기가 가능한 에이전트 워크플로 오케스트레이션

기존 LangChain의 선형적 체인(Chain) 구조는 순차적 작업에는 유용하지만, 반복적인 루프(Loop), 조건부 분기(Conditional Branching), 상태 유지(Statefulness)가 필수적인 복잡한 에이전트 워크플로에는 한계가 있었습니다. LangGraph는 상태를 중심으로 노드 (에이전트) 간의 데이터 흐름을 '그래프'로 모델링하여 이러한 문제를 해결합니다.

LangGraph 핵심 구성 요소

구성 요소	정의 및 역할	'AI 저자 팀' 적용 예시
State (상태)	그래프 전체에서 공유되는 데이터 구조. 애플리케이션의 현재 스냅샷.	messages(대화 기록), task_history(작업 이력), references(참고 문헌) 저장.
Node (노드)	특정 작업을 수행하는 함수 또는 에이전트의 실행 단위.	communicator, content_strategist 등 각 에이전트의 로직 수행.
Edge (엣지)	노드 간의 제어 흐름을 정의하는 연결선.	작업 완료 후 supervisor에서 communicator로의 이동.
Conditional Edge	런타임 데이터에 기반하여 다음 실행 노드를 동적으로 결정하는 분기 로직.	supervisor가 작업 성격에 따라 web_search_agent 또는 content_strategist 를 선택.

1.2 연구의 범위 및 목표

프로덕션 레벨의 에이전트 시스템을 구축하기 위해 필요한 아키텍처 설계 원칙과 구현 디테일을 다룬다. 구체적으로는 다음과 같은 핵심 주제를 탐구한다.

- 1. **상태 관리 전략:** TypedDict 와 Pydantic 을 활용한 강건한 상태 스키마 설계 및 파일 시스템 기반의 지속성(Persistence) 구현 전략.
- 2. **슈퍼바이저 패턴(Supervisor Pattern):** 중앙 집중형 제어 구조를 통한 작업 할당 및 에이전트

간 핸드오프(Handoff) 메커니즘 분석.

3. **지식의 순환 구조:** 웹 검색(Web Search)과 벡터 검색(Vector Search)을 결합하여, 외부 정보를 수집하고 이를 내부 지식베이스(Vector DB)로 내재화하는 자가 발전적(Self-improving) 지식 파이프라인 구축.
4. **구조화된 출력 검증:** LLM의 출력을 제어하기 위한 프롬프트 엔지니어링과 Pydantic 모델을 활용한 출력 파싱 및 에러 핸들링 기법.

2. 기초 아키텍처: 상태 기반 커뮤니케이션 시스템 구축

시작점: 상태 기반의 '커뮤니케이터' 에이전트 구축

역할 정의

커뮤니케이터 (Communicator): 사용자와 시스템 간의 유일한 접점(Frontend Interface). 사용자의 입력을 받고, 다른 에이전트의 작업 결과를 종합하여 보고하는 역할.

핵심 아키텍처: 상태(State) 스키마 설계

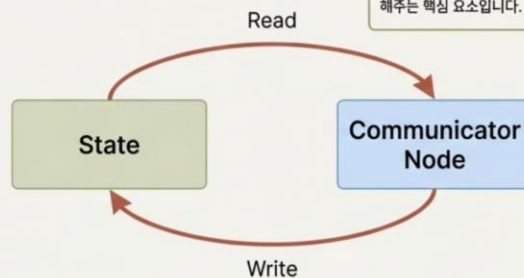
상태는 시스템의 메모리이자 모든 노드가 공유하는 혈액과 같습니다. 초기 설계 시 확장성을 고려하는 것이 매우 중요합니다.

`TypedDict`를 활용하여 강력한 상태 스키마를 정의하고, `AnyMessage` 타입을 통해 다양한 메시지(Human, AI, Tool)를 유연하게 수용합니다.

```
from typing import TypedDict, List
from langchain_core.messages import AnyMessage

class State(TypedDict):
    messages: List[AnyMessage | str]
```

`AnyMessage`의 다형성(Polymorphism)은 서로 다른 역할을 수행하는 에이전트들이 생성하는 이질적인 메시지 형식을 단일 채널로 통합 관리할 수 있게 해주는 핵심 요소입니다.



멀티에이전트 시스템의 가장 기본 단위는 사용자와 상호작용하고 전체적인 진행 상황을 관리하는 인터페이스 에이전트이다. 본 프로젝트에서는 이를 '커뮤니케이터(Communicator)'라고 칭하며, 이 단계에서 LangGraph의 상태 관리와 기본적인 그래프 실행 모델이 수립된다.

2.1 상태 스키마(State Schema)의 설계와 확장성

LangGraph에서 상태(State)는 시스템의 메모리이자 혈액과 같다. 모든 노드는 이 상태를 공유하며 읽고 쓸 수 있다. 초기 단계의 상태 정의는 단순해 보일 수 있으나, 향후 확장성을 고려한 설계가 필수적이다.

```
from typing import TypedDict, List
from langchain_core.messages import AnyMessage
```

```
class State(TypedDict):
    messages: List[AnyMessage | str]
```

위의 코드에서 State 클래스는 TypedDict 를 상속받아 정의되었다. 핵심 필드인 messages 는 List[AnyMessage | str] 타입을 가지는데, 이는 LangChain 의 표준 메시지 타입(HumanMessage, AIMessage, SystemMessage, ToolMessage)뿐만 아니라 일반 문자열까지 수용할 수 있는 유연성을 제공한다.³ 이러한 설계는 다양한 에이전트가 생성하는 이질적인 데이터 형식을 단일 채널로 통합하여 관리할 수 있게 하며, 추후 MemorySaver 와 같은 체크포인트(Checkpointer)가 상태를 직렬화하여 저장할 때 데이터의 무결성을 보장하는 기반이 된다.

AnyMessage 의 다형성과 활용

AnyMessage 타입의 채택은 멀티에이전트 환경에서 특히 중요하다. 각 에이전트는 서로 다른 역할을 수행하며 생성하는 메시지의 성격이 다르다. 예를 들어, 도구(Tool) 사용 결과는 ToolMessage 로, 시스템의 지시는 SystemMessage 로, 에이전트 간의 대화는 AIMessage 로 표현된다. State 가 이러한 다형성(Polymorphism)을 지원함으로써, 개발자는 메시지의 출처나 유형에 구애받지 않고 일관된 방식으로 대화 기록을 처리(append, slice 등)할 수 있다.

2.2 커뮤니케이터 에이전트(Communicator Agent)의 구현 로직

커뮤니케이터는 사용자와 시스템 사이의 접점(Frontend Interface) 역할을 수행한다. 이 에이전트의 주 임무는 사용자의 입력을 받고, 다른 에이전트들의 작업 결과를 종합하여 자연어로 보고하는 것이다.

구현상의 특징적인 부분은 **스트리밍(Streaming) 출력**과 **상태 업데이트** 방식이다.

```
def communicator(state: State):
    #... (시스템 프롬프트 설정)

    gathered = None
    for chunk in system_chain.stream(inputs):
        print(chunk.content, end="") # 실시간 출력

    #... (청크 수집 로직)
```

```
messages.append(gathered)
return {"messages": messages}
```

위 로직에서 `system_chain.stream(inputs)`을 사용하는 것은 사용자 경험(UX) 측면에서 매우 중요하다. LLM 이 긴 응답을 생성할 때, 전체가 완료될 때까지 기다리지 않고 생성되는 즉시 토큰 단위로 출력함으로써 시스템의 반응성을 높인다. 또한, 수집된 전체 메시지(`gathered`)를 리스트에 추가하여 반환함으로써, LangGraph 의 리듀서(Reducer) 기능이 작동하여 전체 상태를 갱신하게 된다.³ 이는 LangGraph 가 노드의 반환값을 받아 상태를 어떻게 병합(Merge)하거나 덮어쓰기(Overwrite)하는지를 보여주는 기본적인 예시이다.

2.3 파일 시스템 기반의 상태 지속성(Persistence) 전략

LangGraph 는 `MemorySaver` 나 `SqliteSaver` 와 같은 내장 체크포인트를 제공하지만, 본 프로젝트에서는 `utils.py` 내의 `save_state` 함수를 통해 파일 시스템에 직접 JSON 형태로 상태를 저장하는 커스텀 방식을 채택하였다.

이 접근 방식의 기술적 함의는 다음과 같다:

1. **가독성(Readability) 및 디버깅:** 바이너리 형태가 아닌 JSON 텍스트로 저장됨으로써, 개발자가 에이전트의 사고 과정과 대화 이력을 직접 열람하고 분석하기 용이하다.
2. **데이터 이식성:** 특정 데이터베이스 시스템에 종속되지 않고, 단순히 파일 복사만으로 상태를 이동하거나 백업할 수 있다.
3. **구현의 단순성:** 복잡한 DB 설정 없이 `os` 와 `json` 라이브러리만으로 지속성을 구현하여 프로토타이핑 속도를 높인다.

저장 로직에서는 `ensure_ascii=False` 를 사용하여 다국어(한국어) 처리를 보장하고, 메시지 객체를 (클래스명, 내용) 튜플로 변환하여 직렬화 가능(Serializable)하게 만드는 전처리 과정이 포함되어 있다. 이는 복잡한 객체를 영구 저장소에 보관할 때 발생할 수 있는 직렬화 오류를 방지하는 방어적 코딩 기법이다.

3. 기능의 분화: 콘텐츠 전략가와 목차 생성 파이프라인

진화 1단계: '콘텐츠 전략가'의 도입과 관심사의 분리

문제점

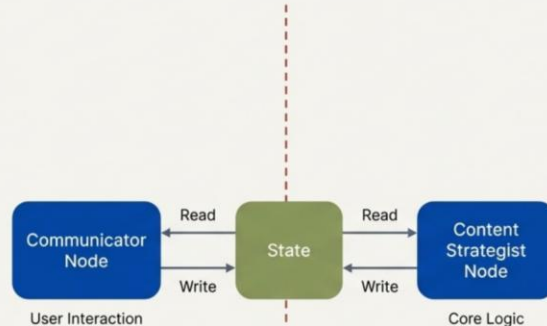
- 단일 '커뮤니케이터 에이전트'가 사용자 소통과 목차 생성이라는 두 가지 복잡한 임무를 동시에 수행하면서 역할이 모호해지고 성능이 저하됩니다.

해결책: 새로운 에이전트의 도입

- 콘텐츠 전략가 (Content Strategist):** 책의 목차(Outline) 생성 및 수정을 전담하는 전문 에이전트.
- 소프트웨어 공학 원칙 적용:** '관심사의 분리(Separation of Concerns)' 원칙을 에이전트 설계에 적용하여 시스템의 모듈성과 유지보수성을 향상시킵니다.

작업 프로세스

- 입력:** 전체 대화 기록(messages)과 현재 목차(outline)를 입력받습니다.
- 수행:** 기존 목차가 있다면 사용자의 새 요구에 맞춰 수정하고, 없다면 새로운 목차를 제안합니다. (반복적 정제, Iterative Refinement)
- 출력:** 수정되거나 새로 생성된 목차 텍스트를 반환합니다.



단일 에이전트가 모든 것을 처리하는 구조를 탈피하기 위해, 책의 목차 작성을 전담하는 '콘텐츠 전략가(Content Strategist)' 에이전트를 도입한다. 이는 소프트웨어 공학의 '관심사의 분리(Separation of Concerns)' 원칙을 에이전트 설계에 적용한 것이다.

3.1 콘텐츠 전략가(Content Strategist)의 역할 정의

content_strategist 노드는 사용자의 요구사항을 분석하여 책의 뼈대가 되는 목차(Outline)를 생성하거나 수정하는 역할을 맡는다. 이 에이전트의 설계에서 주목할 점은 입출력의 구조이다.

- 입력(Input):** 전체 대화 기록(messages)과 현재의 목차(outline).
- 출력(Output):** 수정되거나 새로 생성된 목차 텍스트.

시스템 프롬프트에는 "지난 목차가 있다면 그 버전을 사용자의 요구에 맞게 수정하고, 없다면 새로운 목차를 제안한다"는 지시가 포함되어 있다.³ 이는 에이전트가 백지상태에서 시작하는 것이 아니라, 기존의 컨텍스트를 유지하며 점진적으로 결과물을 발전시키는 **반복적 정제(Iterative Refinement)** 프로세스를 수행함을 의미한다.

3.2 이중 메모리 아키텍처: 상태(State)와 파일(File)의 역할 분담

아키텍처 심층 분석: 이중 메모리 구조 (단기 기억 vs. 장기 기억)

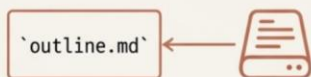
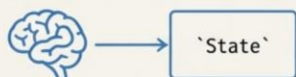
문제점

목차와 같이 크기가 큰 산출물을 `State`의 `messages`에 계속 저장하면, LLM의 컨텍스트 윈도우를 빠르게 소진시켜 비용과 성능 문제를 야기합니다.

해결책: 역할 분담을 통한 메모리 최적화

핵심 산출물은 외부 파일로 영구 저장하고, `State`는 워크플로 제어에 필요한 단기적인 정보만 관리합니다. 이는 RAG와 유사한 원리로, 토큰 효율성을 극대화하는 전략입니다.

단기 기억: 휘발성 대화



장기 기억: 영구적 산출물

메모리 역할 분담

메모리 유형	저장 위치	저장 내용	역할 및 특징
단기 기억	State["messages"]	대화 로그, 작업 지시, 에이전트 간 알림	워크플로 흐름 제어 및 즉각적인 컨텍스트 제공. 휘발성.
장기 기억	`data/outline.md`	생성된 목차 본문	에이전트가 생성한 핵심 산출물. 영구적이며 지속적으로 업데이트.

에이전트의 단기 기억(Short-term Memory)과 장기 기억(Long-term Memory)을 구분하여 관리하는 독특한 아키텍처를 보여준다.

목차 내용을 State 내의 메시지로만 관리할 경우, 목차의 길이가 길어짐에 따라 LLM의 컨텍스트 윈도우를 과도하게 점유하는 문제가 발생할 수 있다. 따라서 목차 본문은 별도의 마크다운 파일(outline.md)로 저장하고(save_outline), 필요할 때만 읽어와(get_outline) 프롬프트에 주입하는 방식을 취한다. 이는 RAG 시스템에서 문서를 외부 저장소에 두고 검색하는 원리와 유사하며, 토큰 효율성을 극대화하는 전략이다.

3.3 에이전트 간 중복 통신 방지 및 최적화

초기 구현 단계에서 content_strategist가 목차를 생성한 후, 그 결과를 communicator에게 전달하면 communicator가 이를 다시 사용자에게 앵무새처럼 반복하여 출력하는 비효율이 관찰되었다. 이는 멀티에이전트 시스템에서 흔히 발생하는 '정보 중복(Information Redundancy)' 문제이다.

이를 해결하기 위해 communicator의 시스템 프롬프트를 다음과 같이 수정하였다:

"사용자도 outline(목차)을 이미 보고 있으므로, 다시 출력할 필요는 없다."

이러한 프롬프트 엔지니어링은 에이전트 간의 역할 경계를 명확히 하고, 불필요한 토큰 생성을 억제하여 시스템의 응답 속도와 경제성을 높이는 튜닝 과정이다. 에이전트 간 핸드오프(Handoff) 시에는 단순한 데이터 전달뿐만 아니라, '무엇을 말하고 무엇을 말하지 말아야 하는지'에 대한 메타 인지적 지시가 동반되어야 함을 시사한다.

4. 중앙 제어와 자율성의 조화: 슈퍼바이저(Supervisor) 패턴

진화 2단계: '슈퍼바이저' 패턴 도입과 중앙 제어

새로운 도전 과제

에이전트의 수가 늘어남에 따라, 어떤 에이전트가 언제 작업을 수행해야 할지 결정하는 중앙 제어 장치가 필요해집니다.

해결책: 슈퍼바이저(Supervisor) 아키텍처

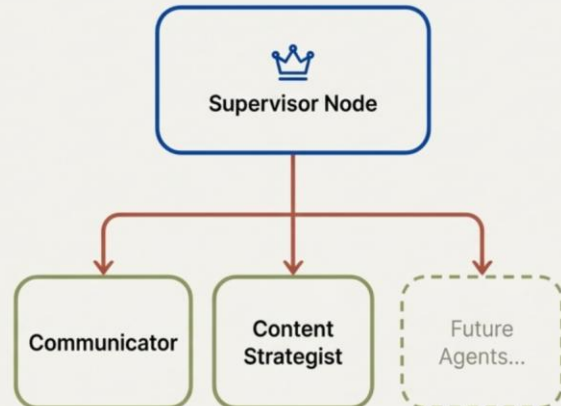
슈퍼바이저는 작업을 직접 수행하지 않고, 사용자의 요청을 분석하여 가장 적절한 하위 에이전트(Worker)에게 작업을 위임하는 라우터(Router) 역할을 수행합니다. (예: 잡지사 편집장, 프로젝트 매니저)

이를 통해 시스템의 모듈성과 확장성이 극대화됩니다. 새로운 기능(에이전트)이 추가되어도 슈퍼바이저의 선택지에만 등록하면 되기 때문입니다.

LangGraph 구현

조건부 엣지(Conditional Edge)를 통해 구현됩니다.

슈퍼바이저 노드는 다음에 실행할 노드의 이름(예: 'content_strategist')을 반환하고, 그래프는 이 결과에 따라 동적으로 다음 경로를 결정합니다.



시스템의 기능이 확장됨에 따라 어떤 에이전트가 언제 작업을 수행해야 할지 결정하는 중앙 제어 장치가 필요해진다. LangGraph 의 **슈퍼바이저 아키텍처(Supervisor Architecture)**는 복잡한 워크플로를 관리하는 가장 강력한 패턴 중 하나이다.

4.1 슈퍼바이저 에이전트의 의사결정 메커니즘

슈퍼바이저는 작업을 직접 수행하지 않고, 사용자의 요청을 분석하여 적절한 하위 에이전트(Worker)에게 작업을 위임하는 라우터(Router) 역할을 한다. 이는 대학 조별 과제의 조장이나 잡지사의 편집장과 같은 역할이다.

LangGraph 에서 슈퍼바이저는 **조건부 엣지(Conditional Edge)**를 통해 구현된다. 슈퍼바이저 노드의 실행 결과는 다음 실행할 노드의 이름(예: "content_strategist", "communicator")이 되며, supervisor_router 함수는 이 결과값을 바탕으로 그래프의 흐름을 분기시킨다.

```
graph_builder.add_conditional_edges(  
    "supervisor",  
    supervisor_router,  
    {  
        "content_strategist": "content_strategist",
```

```

    "communicator": "communicator",
    "web_search_agent": "web_search_agent",
    #...
}
)

```

이러한 구조는 시스템의 모듈성을 극대화한다. 새로운 기능을 추가하려면 해당 기능을 수행하는 에이전트 노드를 만들고, 슈퍼바이저의 선택지에 추가하기만 하면 된다. 이는 **확장 가능한(Scalable)** 시스템 설계의 핵심이다.

4.2 구조화된 출력(Structured Output)을 통한 제어권 강화

슈퍼바이저의 두뇌: Pydantic을 이용한 구조화된 출력 제어

라우터로서의 LLM의 위험성

슈퍼바이저 LLM이 'content_strategist'와 같은 정확한 노드 이름 대신 '목차를 만드는 에이전트가 좋겠어' 같은 자연어를 반환하면, 그래프 실행이 중단되는 심각한 오류가 발생합니다.

해결책: 출력 스키마 강제 (Structured Output)

- **'Pydantic 모델'**: LLM의 출력이 따라야 할 데이터 구조(스키마)를 코드로 명확하게 정의합니다.
- **'llm.with_structured_output(Task)'**: LangChain의 이 기능을 사용하면, LLM이 내부적으로 함수 호출(Tool Calling)이나 JSON 모드를 활용하여 반드시 Pydantic 스키마에 맞는 형식으로만 출력하도록 강제할 수 있습니다.

```

# models.py
class Task(BaseModel):
    agent: Literal["content_strategist", "communicator", "web_search_agent"]
    done: bool = Field(..., description="종료 여부")
    description: str = Field(..., description="어떤 작업을 해야 하는지에 대한 설명")

```

- **agent: Literal[...]**: 선택 가능한 에이전트의 범위를 코드 레벨에서 제한하여 라우팅 오류를 원천적으로 차단합니다.
- **done: bool**: 작업 완료 여부를 추적합니다.
- **description: str**: 하위 에이전트에게 '누가' 할지뿐만 아니라 '무엇을' 해야 하는지에 대한 구체적인 작업 지시를 전달하여 협업의 정확도를 높입니다.

LLM 을 라우터로 사용할 때 가장 큰 위험은 출력의 비일관성이다. 예를 들어, "content_strategist"라는 정확한 노드 이름 대신 "목차를 만드는 에이전트가 필요해 보입니다"와 같은 자연어 문장을 반환하면 그래프 실행이 중단된다.

이를 해결하기 위해 본 프로젝트에서는 **Pydantic** 모델을 활용한 구조화된 출력을 도입하였다.

Task 모델의 정의와 검증

models.py 에 정의된 Task 클래스는 슈퍼바이저의 출력을 엄격하게 제어한다.

```

class Task(BaseModel):
    agent: Literal["content_strategist", "communicator", "web_search_agent", "vector_search_agent"]
    = Field(...)

    done: bool = Field(..., description="종료 여부")

    description: str = Field(..., description="어떤 작업을 해야 하는지에 대한 설명")

    #...

```

1. **agent 필드:** Literal 타입을 사용하여 선택 가능한 에이전트의 범위를 코드 레벨에서 제한한다. 이는 런타임 라우팅 오류를 원천적으로 차단한다.
2. **description 필드:** 슈퍼바이저가 하위 에이전트에게 구체적인 작업 지시 사항을 전달하는 통신 채널이다. 단순히 "누가" 할 것인지 뿐만 아니라 "무엇을" 해야 하는지까지 명시함으로써 에이전트 간 협업의 정확도를 높인다.
3. **llm.with_structured_output(Task):** LangChain 의 이 기능을 사용하면 LLM 이 내부적으로 함수 호출(Tool Calling)이나 JSON 모드를 사용하여 Pydantic 스키마에 부합하는 출력을 생성하도록 강제한다.

4.3 작업 이력(Task History) 기반의 상태 추적

시스템의 메타 인지: '작업 이력'을 통한 상태 추적

도입 배경
슈퍼바이저 아키텍처가 도입되면서, 시스템이 지금까지 어떤 작업을 수행해왔는지 추적하고 공유할 필요성이 생겼습니다.

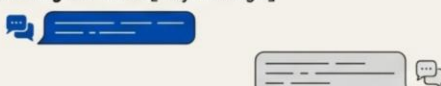
`State` 스키마 확장
`task\_history: List[Task]` 필드를 `State`에 추가합니다. 이 리스트에는 슈퍼바이저가 생성한 모든 `Task` 객체가 시간 순서대로 기록됩니다.

'작업 이력'의 역할

1. **현재 작업 검증:** 각 에이전트는 실행 시 `task\_history`의 마지막 항목을 확인하여 자신에게 할당된 작업이 맞는지 검증합니다. (방어적 코딩)
2. **문맥 공유:** 이후 실행되는 에이전트들은 `task\_history`를 통해 이전 작업들을 파악하고, 전체 프로젝트 흐름 속에서 자신의 역할을 인지합니다.
3. **완료 처리:** 에이전트는 작업이 끝나면 해당 `Task`의 `done` 상태를 `True`로 업데이트하여 작업 완료를 명시적으로 기록합니다.

State Object

messages: List[AnyMessage]



task_history: List[Task]

{ agent: "analyst_agent",
description: "Analyze initial data...",
done: true }

↓

{ agent: "planner_agent",
description: "Generate project plan...",
done: true }

↓

{ agent: "writer_agent",
description: "Draft first section...",
done: true }

↓

{ agent: "reviewer_agent",
description: "Review draft content...",
done: false }

슈퍼바이저 아키텍처의 도입과 함께 State 에는 task_history 리스트가 추가되었다. 이는 시스템의

메타 인지(Meta-cognition) 능력을 구현한 것이다.

- **현재 작업 확인:** 각 에이전트는 실행 시 task_history 의 마지막 항목을 확인하여 자신에게 할당된 작업이 맞는지 검증한다. 이는 잘못된 라우팅으로 인한 오작동을 방지하는 방어적 로직이다.
- **완료 처리:** 작업이 끝나면 done=True 로 상태를 업데이트하고 완료 시각을 기록한다.
- **문맥 공유:** 이후 실행되는 에이전트들은 task_history 를 통해 이전에 어떤 작업들이 수행되었는지 파악하고, 전체 프로젝트의 흐름 속에서 자신의 위치를 인지할 수 있다.³

5. 지식의 확장: RAG 기반 벡터 검색 에이전트

진화 3단계: RAG 도입으로 외부 지식과 연결하기

근본적인 한계

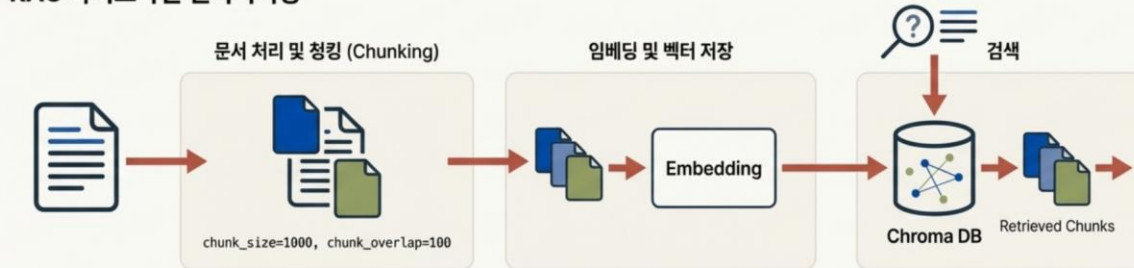
지금까지의 시스템은 내부적으로 주어진 정보만으로 콘텐츠를 생성했습니다.

이는 환각(Hallucination)에 취약하며 최신 정보를 반영할 수 없습니다.

해결책: 검색 증강 생성 (Retrieval-Augmented Generation, RAG)

외부 지식 소스(웹, 문서)에서 관련 정보를 검색(Retrieval)하고, 이를 LLM의 컨텍스트에 주입하여 답변을 생성(Generation)하는 기술.

RAG 파이프라인 엔지니어링



AI가 작성한 목차의 신뢰성을 확보하기 위해서는 할루시네이션을 방지하고 사실에 입각한 정보가 필요하다. 이를 위해 **검색 증강 생성(Retrieval-Augmented Generation, RAG)** 시스템을 통합한다.

5.1 RAG 파이프라인의 엔지니어링

RAG 시스템은 단순한 검색이 아니라, 데이터의 수집(Extract), 변환(Transform), 적재(Load) 과정을 포함하는 ETL 파이프라인으로 구성된다.

문서 처리와 청킹(Chunking) 전략

tools.py에 구현된 split_documents 함수는 RecursiveCharacterTextSplitter를 사용한다.

```
text_splitter = RecursiveCharacterTextSplitter(  
    chunk_size=1000,  
    chunk_overlap=100  
)
```

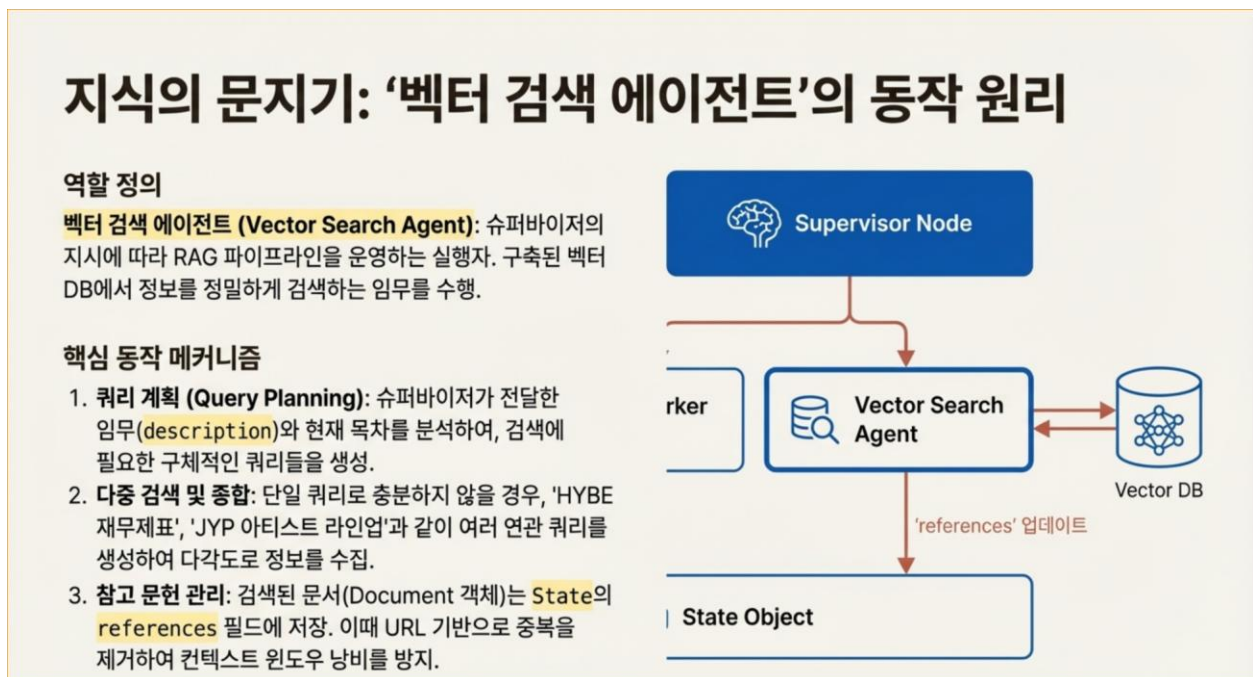
- **청크 크기(1000 자):** 너무 작으면 문맥이 잘리고, 너무 크면 검색 정밀도가 떨어지며 임베딩 모델의 토큰 제한에 걸릴 수 있다. 1000 자는 문단 단위의 의미를 보존하기에 적절한 크기이다.

- **중복(Overlap 100 자):** 청크 간의 경계에서 정보가 유실되는 것을 방지하기 위해 10% 정도의 중복 구간을 둔다. 이는 검색된 청크들을 연결하여 답변을 생성할 때 문맥의 연속성을 보장한다.

벡터 저장소와 임베딩의 지속성

Chroma DB 를 사용하여 임베딩된 벡터를 로컬 디스크(data/chroma_store)에 영구 저장한다. 이는 프로세스가 재시작되더라도 구축된 지식 베이스가 유지되도록 한다. 특히, documents_to_chroma 함수에는 이미 저장된 문서의 URL 을 확인하여 중복 저장을 방지하는 로직이 포함되어 있다. 이는 벡터 DB 의 용량을 최적화하고 검색 결과의 중복을 막아 랭킹(Ranking) 품질을 유지하는 데 필수적이다.

5.2 벡터 검색 에이전트(Vector Search Agent)의 동작 원리



vector_search_agent 는 RAG 시스템의 운영자이다. 슈퍼바이저가 정보 검색을 지시하면, 이 에이전트는 다음과 같은 과정을 수행한다.

1. **쿼리 계획(Query Planning):** 슈퍼바이저가 전달한 임무(mission)와 현재 목차 상황을 분석하여, 검색에 필요한 구체적인 쿼리들을 생성한다. 에이전트는 .bind_tools([retrieve])를 통해 검색 도구를 자유롭게 호출할 수 있다.
2. **다중 검색 및 종합:** 단일 쿼리로 모든 정보를 얻을 수 없는 경우, 에이전트는 여러 개의 연관

쿼리(예: "HYBE 재무제표", "JYP 아티스트 라인업")를 생성하여 다각도로 정보를 수집한다.

3. **참고 문헌 관리:** 검색된 결과(Document 객체)는 State 의 references 필드에 저장된다. 이때 URL 기반의 중복 제거 로직을 수행하여, 동일한 정보가 중복되어 컨텍스트 윈도우를 낭비하는 것을 방지한다.

이 과정에서 State 객체의 references 딕셔너리는 시스템이 수집한 '지식의 저장소' 역할을 하며, 향후 콘텐츠 전략가가 목차를 작성할 때 근거 자료로 활용된다.

6. 실시간 정보의 통합: 웹 검색 에이전트와 자가 발전 루프

진화 4단계: '웹 검색 에이전트'로 실시간 정보 통합하기

남아있는 문제

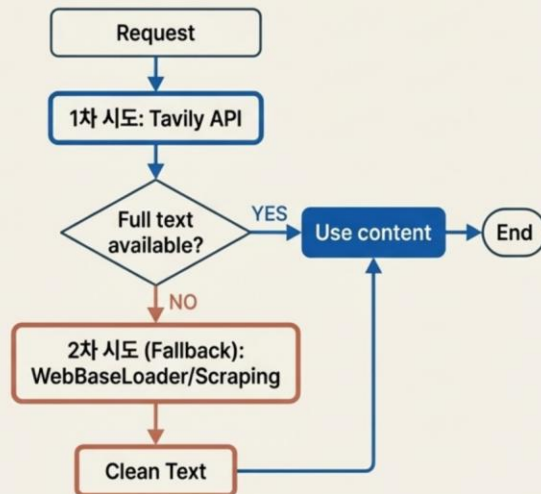
벡터 DB에 저장된 정보는 구축된 시점에 고정되어 있습니다. 최신 뉴스나 실시간 기업 정보와 같은 동적인 데이터를 다루기에는 한계가 있습니다.

해결책: 실시간 웹 검색 기능 추가

웹 검색 에이전트 (Web Search Agent): 인터넷을 실시간으로 검색하여 최신 정보를 수집하는 에이전트.

하이브리드 정보 수집 전략 (Fallback Mechanism)

정보 수집의 성공률과 강건성(Robustness)을 극대화하기 위한 이중 접근 방식.



벡터 DB에 저장된 정보만으로는 최신 트렌드나 특정 기업의 실시간 정보를 다루기에 한계가 있다. 이를 보완하기 위해 인터넷을 실시간으로 검색하는 **웹 검색 에이전트(Web Search Agent)**가 도입된다.

6.1 Tavily API와 웹 스크래핑의 하이브리드 접근

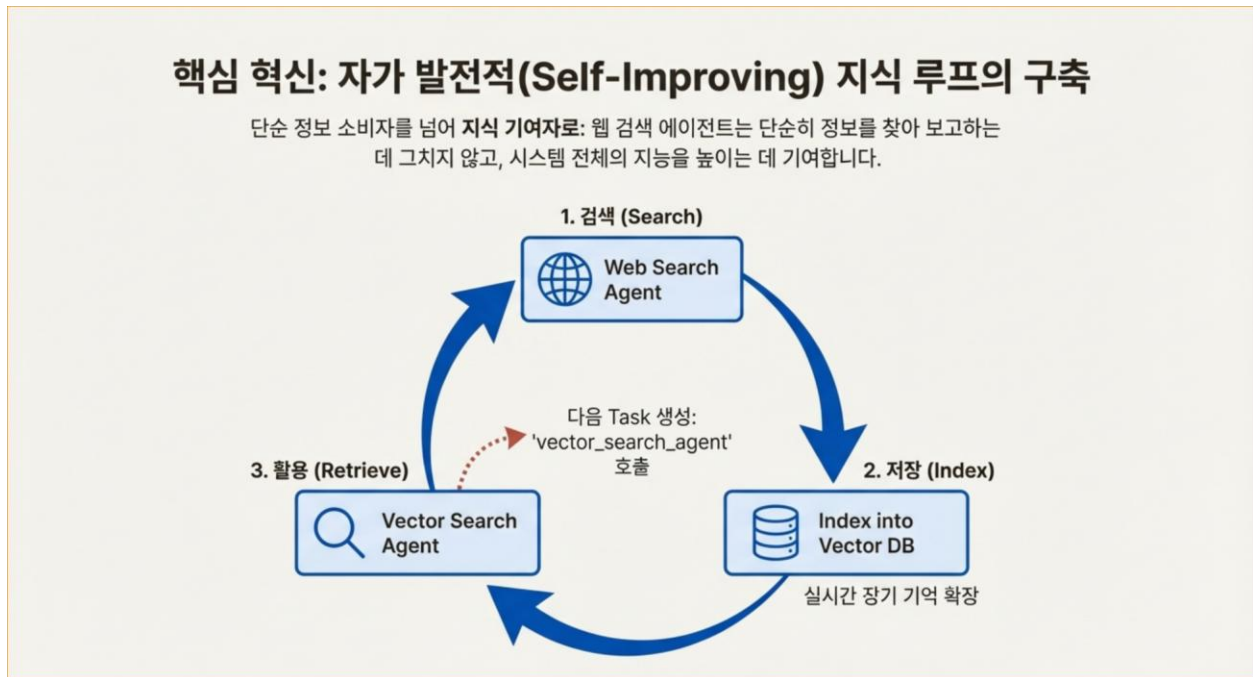
본 시스템은 **Tavily Search API**를 주 검색 도구로 사용한다. Tavily는 LLM에 최적화된 검색 결과를 제공하지만, 때로는 웹 페이지의 전문(Full-text)을 가져오지 못하는 경우가 있다.

이를 해결하기 위해 web_search 도구는 **폴백(Fallback) 메커니즘**을 구현하고 있다.

1. **1차 시도:** Tavily API를 통해 검색 결과와 raw_content를 요청한다.
2. **2차 시도 (실패 시):** raw_content가 비어있는 경우, WebBaseLoader를 사용하여 해당 URL에 직접 접근, 내용을 스크래핑한다.
3. **후처리:** 스크래핑된 텍스트에서 불필요한 공백과 개행 문자를 제거하여 정제한다.

이러한 하이브리드 접근은 정보 수집의 성공률(Robustness)을 극대화하며, 다양한 형태의 웹 소스에 대응할 수 있게 한다.

6.2 자가 발전적(Self-Improving) 지식 루프 구축



웹 검색 에이전트의 가장 혁신적인 특징은 '검색-저장-재검색'의 순환 루프를 형성한다는 점이다.

1. **검색(Search):** 웹 검색을 통해 최신 정보를 수집한다.
2. **저장(Index):** 수집된 정보를 즉시 벡터 DB(Chroma)에 임베딩하여 저장한다(add_web_pages_json_to_chroma). 이 과정은 시스템의 장기 기억을 실시간으로 확장하는 행위이다.
3. **활용(Retrieve):** 웹 검색 에이전트는 작업을 마친 후, 스스로 vector_search_agent 에게 바통을 넘기는 Task 를 생성한다. 이제 벡터 DB 에는 방금 수집한 따끈따끈한 정보가 들어있으므로, 벡터 검색 에이전트는 이를 정밀하게 검색하여 활용할 수 있게 된다.

이 메커니즘은 시스템이 사용될수록 지식 베이스가 풍부해지는 강화학습적 특성을 모사한다. 에이전트는 단순히 정보를 소비하는 주체가 아니라, 지식을 생산하고 축적하여 시스템 전체의 능력을 높이는 기여자(Contributor)가 된다.

6.3 동적 워크플로의 완성

최종적으로 완성된 그래프는 다음과 같은 동적 흐름을 갖는다.

- **Supervisor** -> (정보 부족 판단) -> **Web Search Agent** -> (웹 검색 및 DB 저장) -> (Task 생성) -> **Vector Search Agent** -> (DB 검색 및 Reference 업데이트) -> **Communicator**

이 과정에서 에이전트들은 서로의 상태(task_history, references)를 공유하며 유기적으로 협력한다. 웹 검색 에이전트가 생성한 Task의 description에는 검색했던 쿼리 정보가 포함되어 있어, 이어받는 벡터 검색 에이전트가 무엇을 검색해야 할지 정확하게 인지할 수 있게 한다.

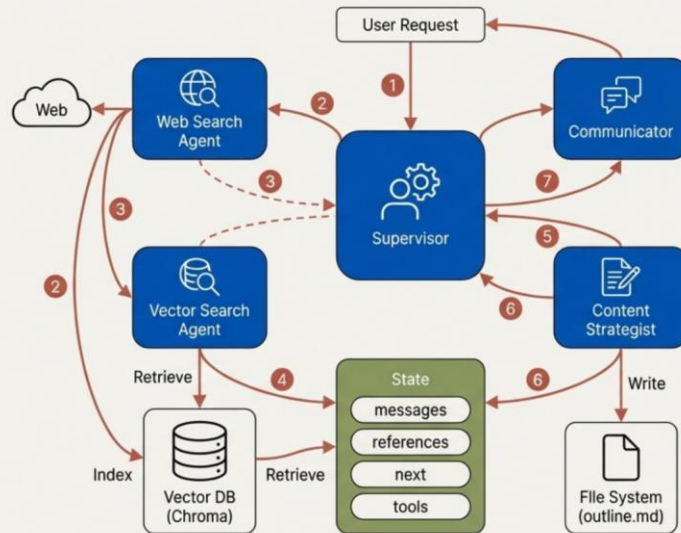
완성된 아키텍처: 동적 워크플로의 실제 작동 방식

사용자 시나리오

사용자가 '최신 엔터테인먼트 산업 동향을 반영하여 'K-POP의 미래'라는 책의 목차를 만들어줘' 라고 요청.

실행 경로 (Execution Path)

1. **'Supervisor'**: 요청을 분석하고, 정보가 부족하다고 판단하여 'web_search_agent' 호출.
2. **'Web Search Agent'**: 웹을 검색하여 HYBE, JYP 등의 최신 기사를 수집. 수집한 정보를 벡터 DB에 저장(Index).
3. **'Web Search Agent'**: 작업 완료 후, 'vector_search_agent'를 호출하는 새로운 'Task' 생성.
4. **'Vector Search Agent'**: 벡터 DB에서 관련 정보를 정밀 검색하여 'State'의 'references'에 추가.
5. **'Supervisor'**: 이제 정보가 충분하다고 판단하고, 'content_strategist' 호출.
6. **'Content Strategist'**: 'references'를 참조하여 목차를 작성하고 파일에 저장.
7. **'Communicator'**: 최종적으로 사용자에게 작업 완료를 보고.



7. 심화 분석 및 향후 발전 방향

본 프로젝트를 통해 구축된 멀티에이전트 시스템은 LangGraph의 강력한 기능들을 실증적으로 보여주었다. 하지만 프로덕션 환경에서의 안정성과 성능을 더욱 높이기 위해서는 다음과 같은 고급 아키텍처 패턴의 도입을 고려해야 한다.

7.1 CRAG (Corrective RAG) 패턴의 도입 가능성

현재 시스템은 검색된 문서의 품질을 맹목적으로 신뢰한다. 하지만 검색 결과가 부정확하거나 관련성이 떨어질 수 있다. 이를 보완하기 위해 **CRAG** 패턴을 적용할 수 있다.

- **평가자(Evaluator) 노드 추가:** 검색된 문서가 사용자의 질문과 관련이 있는지 평가하는 노드를 추가한다.
- **경로 분기:**
 - *Correct:* 관련성이 높으면 바로 답변 생성.
 - *Incorrect:* 관련성이 없으면 웹 검색을 다시 수행.
 - *Ambiguous:* 모호하면 지식을 정제하거나 추가 검색 수행.이러한 평가 루프는 할루시네이션을 획기적으로 줄이고 시스템의 신뢰도를 높이는 핵심 기술이 될 것이다.

7.2 테스트 및 검증(CI/CD) 파이프라인

에이전트 시스템은 비결정론적(Non-deterministic) 특성을 가지므로 테스트가 어렵다. 이를 극복하기 위해 **LangSmith**와 **Pytest**를 활용한 자동화된 테스트 파이프라인 구축이 필수적이다.

- **단위 테스트:** 각 도구(Tool)와 노드가 예상대로 입출력을 처리하는지 검증한다.
- **통합 테스트:** 에이전트 간의 핸드오프와 시나리오별 그래프 실행 경로가 의도한 대로 흐르는지 시뮬레이션한다.
- **회귀 테스트:** 프롬프트 수정 후 성능이 저하되지 않는지 지속적으로 모니터링한다.

7.3 스웜(Swarm) 아키텍처와의 비교 및 융합

본 시스템은 중앙의 슈퍼바이저가 제어하는 계층적 구조를 띠고 있다. 이는 통제가 용이하고 흐름이 명확하다는 장점이 있다. 반면, 에이전트들이 서로 직접 소통하며 협력하는 **스웜 아키텍처**는 더 높은 유연성을 제공할 수 있다. 향후 시스템이 더욱 복잡해진다면, 슈퍼바이저가 큰 방향만 잡고 세부적인 협업은 에이전트끼리 자율적으로 수행하는 하이브리드 모델로의 진화를 모색해볼 수

있다.

8. 결론

LangGraph를 활용한 멀티에이전트 시스템 구축은 단순한 기술적 조립을 넘어, AI 에이전트들에게 역할과 책임을 부여하고 그들 간의 사회적 상호작용을 설계하는 과정이다. 분석한 커뮤니케이터, 콘텐츠 전략가, 슈퍼바이저, 그리고 검색 에이전트들의 유기적인 협업 모델은 복잡한 지식 작업을 자동화하려는 모든 시도에 있어 강력한 참조 모델(Reference Model)이 될 것이다. 상태 관리의 정교함, 구조화된 통신의 엄격함, 그리고 지식 확장의 유연성이 결합될 때, 비로소 인간과 협력할 수 있는 진정한 의미의 'AI 팀'이 탄생한다.